

Security Audit

PoolParty 3rd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	32
Conclusion	33
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The PoolParty team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The PoolParty™ Exchange & Marketplace is positioning itself as an innovative decentralized trading platform (DEX) that aims to eliminate the common cost of "price impact" in crypto trading. By integrating an over-the-counter (OTC) marketplace directly into the DEX structure, PoolParty™ enables sellers—especially large holders—to execute trades without affecting market prices, resulting in higher profits for them and attractive discounts for buyers. The platform is styled around the tagline "Let the chart fly", underscoring its goal of seamless, impact-free crypto trading.

Project Name: PoolParty

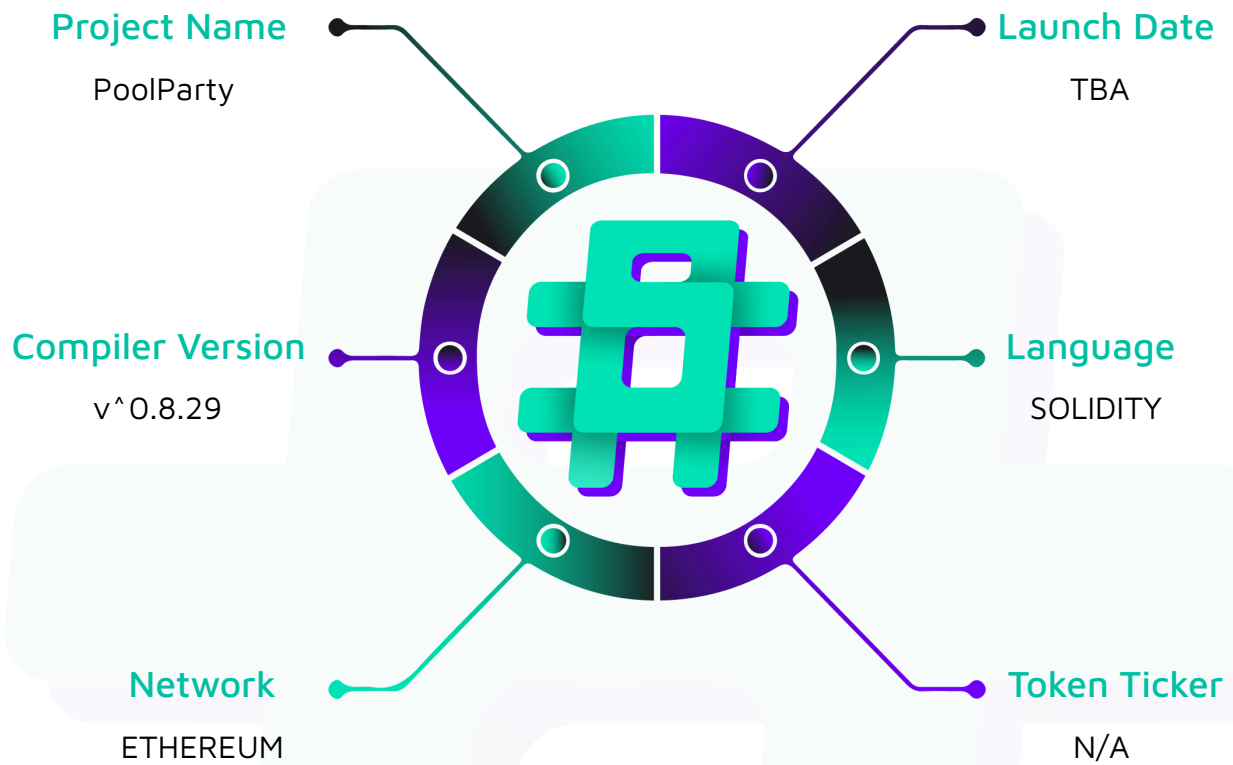
Project Type: dApp

Compiler version: v^0.8.29

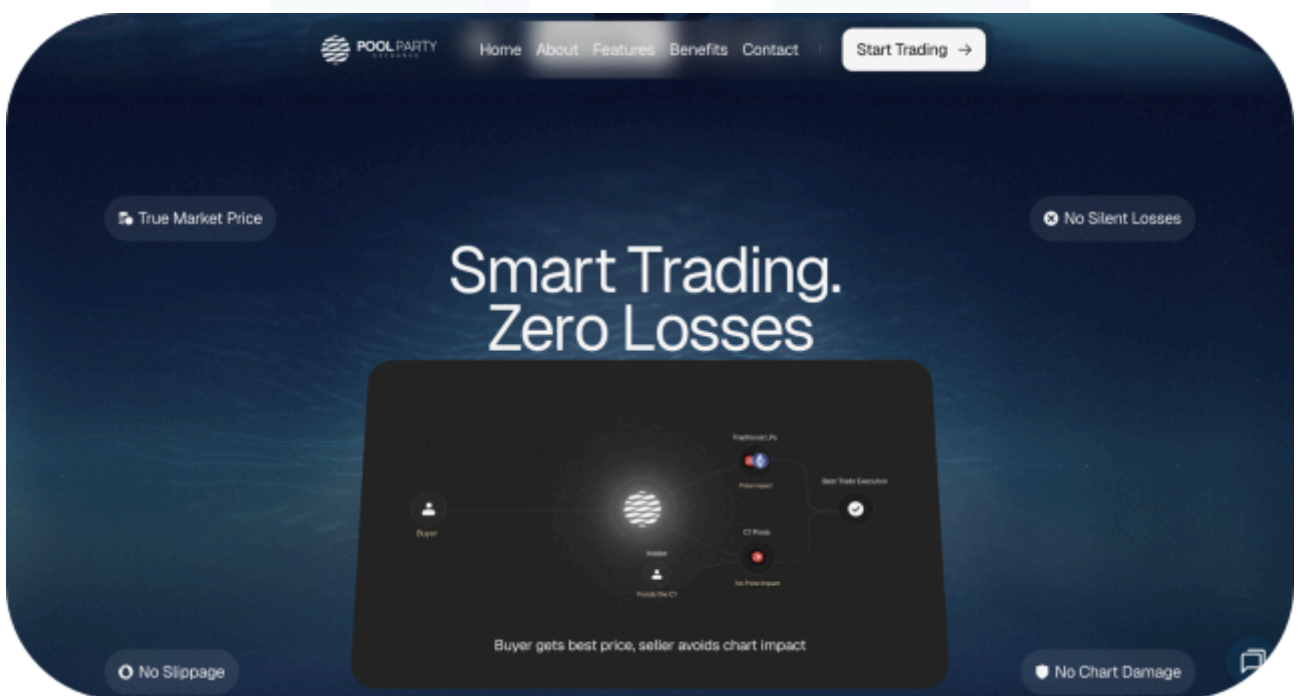
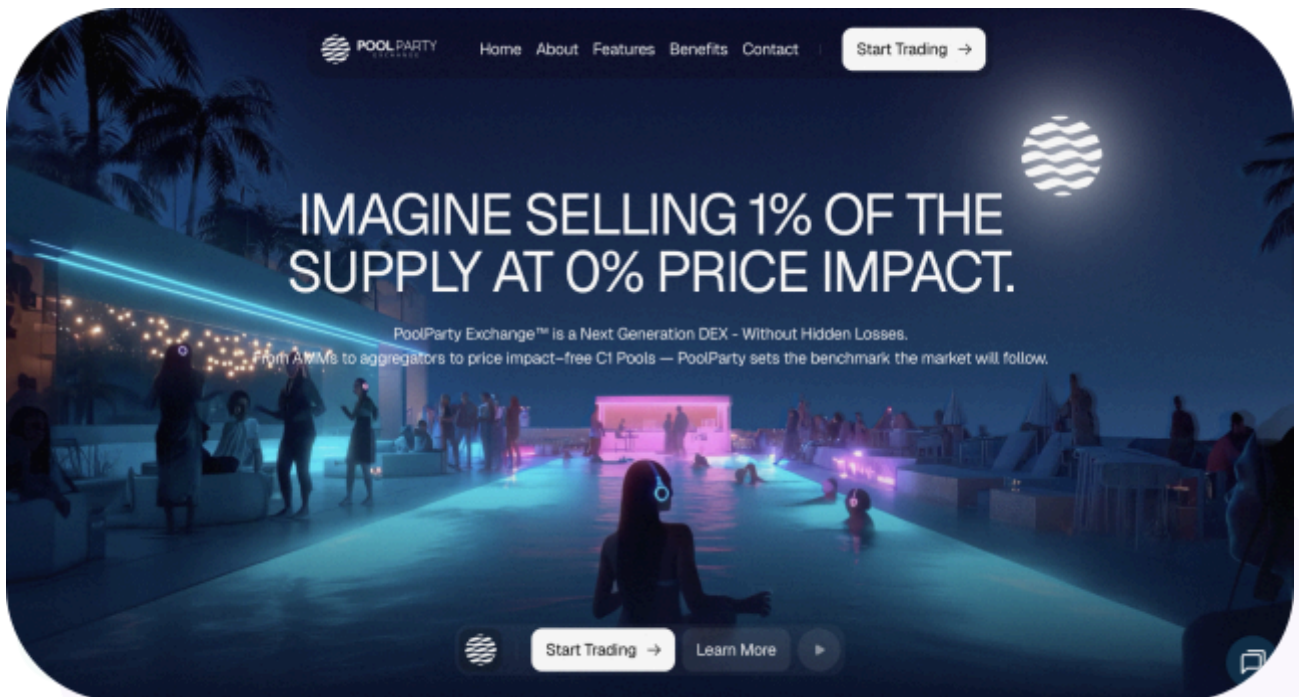
Website: <https://www.poolparty.market/>

Logo:



Visualised Context:

Project Visuals:



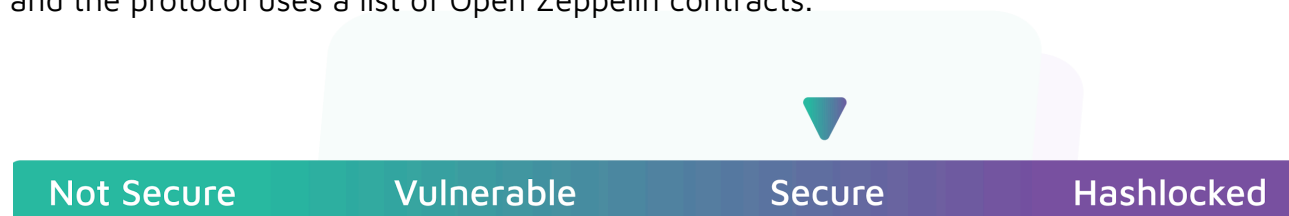
Audit Scope

We at Hashlock audited the solidity code within the PoolParty project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PoolParty Smart Contracts
Platform	Ethereum / Solidity
Audit Date	December, 2025
Contract 1	publicPool.sol
Contract 2	ownedPool.sol
Contract 3	PoolPartySwap.sol
Contract 4	IPublicPoolContract.sol
Contract 5	IOwnedPoolContract.sol
Contract 6	UpdatePoolPartyStruct.sol
Contract 7	PoolPartyLogic.sol
Contract 8	PoolPartyFrenlyBridge.sol
Audited GitHub Commit Hash	63a9c0de9e79e35a689304393b931e169ab4db1f
Fix Review GitHub Commit Hash	d467e481b24dfbf4f242dd0f0a81d184b5eb8a8a

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

5 Medium severity vulnerabilities

9 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
PoolPartySwap.sol - Core contract managing pools, feels, and platform functionalities	Contract achieves this functionality.
OwnedPool.sol - Implement Single-Owner token pools with optional Vesting	Contract achieves this functionality.
PublicPool.sol - Implement many-to-many token pools with optional Vesting	Contract achieves this functionality.
PoolPartyFrenlyBridge.sol - Serves as a Bridge between End Users and Protocol (Vaults and Pools)	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the PoolParty project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PoolParty project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PublicPool&OwnedPool#takeFee - Arithmetic underflow in fee distribution causes all purchases to revert (DoS)

Description

Both `ownedPool` and `publicPool` compute fees in `takeFee()` using different denominators and unchecked subtraction that can underflow. In `ownedPool`, `partyFeeAmount` is computed as $(\text{price} * \text{partyFee}) / 100$, while `projectFee` and `resellerFee` are computed as basis points of the full price: $(\text{price} * \text{projectFee}) / 10000$ and $(\text{price} * \text{resellerFee}) / 10000$. The code then performs `partyFeeAmount -= (projectFee + resellerFee + resellerCommunityFee)` inside an unchecked block.

If the sum of `projectFee + resellerFee*2` (reseller community fee equals `resellerFee`) exceeds `partyFeeAmount`, the subtraction underflows, resulting in a massive value. The subsequent `call{value: partyFeeAmount}` attempts to send this huge amount and reverts with insufficient balance, causing the entire `poolTransfer` to fail. The same issue applies to `publicPool` with a slightly different denominator for `partyFeeAmount`.

Proof of Concept

An example of a test that simulates underflow is shown below.

```
// SPDX-License-Identifier: MIT

pragma solidity 0.8.29;

import "forge-std/Test.sol";
import "../contracts/ownedPool.sol";

import {MockERC20} from "../test/mocks/MockERC20.sol";
import {MockPoolPartyMain} from "../test/mocks/MockPoolPartyMain.sol";

contract FeeUnderflowTest is Test {
```



```

ownedPool pool;

MockERC20 token;

MockERC20 weth;

MockPoolPartyMain poolParty;

address attacker = address(0xdead);

address owner = address(0xbeef);

function setUp() public {

    token = new MockERC20("Test", "TST", 18);

    weth = new MockERC20("WETH", "WETH", 18);

    poolParty = new MockPoolPartyMain();

    pool = new ownedPool();

    pool.initialize(address(token), address(0), 0, address(poolParty), owner,
address(weth), 0);

    token.mint(address(pool), 10000 ether);

    pool.updateAmounts(10000 ether, 0, 2);

    // High project + reseller fees

    poolParty.setFeeData(1000, payable(owner), true, "", payable(owner),
payable(owner), 500);

    bytes32 POOLPARTY_ADMIN = keccak256("IS_ADMIN");

    vm.prank(address(poolParty));

    pool.grantRole(POOLPARTY_ADMIN, address(this));

}

```

```

function testFeeUnderflow() public {
    vm.deal(address(this), 10 ether);
    vm.expectRevert();
    pool.poolTransfer{value: 1 ether}(
        payable(attacker),
        1000 ether,
        "",
        2,
        address(0x1),
        payable(owner)
    );
}
}

```

Impact

Any pool with aggregate project/reseller/community fees exceeding the platform party fee becomes completely unbuyable. This is a hard denial-of-service for purchases from the affected pool and can be triggered accidentally via misconfiguration or maliciously via governance.

Recommendation

Remove the unchecked arithmetic and enforce that $(\text{projectFee} + \text{resellerFee} + \text{resellerCommunityFee}) \leq \text{partyFeeAmount}$, or compute a single $\text{totalFee} = \text{partyFeeAmount} + \text{projectFee} + \text{resellerFee} + \text{resellerCommunityFee}$ from price and require $\text{totalFee} \leq \text{price}$. Then set $\text{newPrice} = \text{price} - \text{totalFee}$. Standardize units across pools and document intended fee semantics.

Status

Resolved

[H-02] PublicPool#takeFee - Public pool's partial-fill refund miscalculates returnETH, causing inevitable revert on gas exhaustion

Description

`takeFee(msg.value)` transfers fees immediately and returns `newPrice = msg.value - totalFees`. If the gas sentinel breaks the loop, the code sets `params.returnETH = msg.value - params.totalETHSent`, but the contract only holds `newPrice` (fees already sent). After paying the sellers' `params.totalETHSent`, refunding `msg.value - params.totalETHSent` exceeds the remaining balance (`newPrice - params.totalETHSent`) and the refund reverts, failing the entire transaction.

Proof of Concept

An example of a test that simulates underflow is shown below.

```
// SPDX-License-Identifier: MIT

pragma solidity 0.8.29;

import "forge-std/Test.sol";
import "../contracts/publicPool.sol";
import {MockERC20} from "../test/mocks/MockERC20.sol";

contract PublicPoolRefundTest is Test {

    publicPool pool;

    MockERC20 token;

    MockERC20 weth;

    address poolParty = address(0xdead);

    address buyer = address(0xb0b);

    address feeReceiver = address(0xfee);

    function setUp() public {

        token = new MockERC20("Test", "TST", 18);
```

```

weth = new MockERC20("WETH", "WETH", 18);

pool = new publicPool();

pool.initialize(address(token), poolParty, address(weth));

token.mint(address(pool), 10000 ether);

bytes32 POOLPARTY_ADMIN = keccak256("IS_ADMIN");

vm.prank(poolParty);

pool.grantRole(POOLPARTY_ADMIN, address(this));

pool.updatePublicPoolAmounts(address(0x1), 10000 ether);

for (uint i = 0; i < 50; i++) {
    token.mint(address(pool), 100 ether);
    pool.contribute(100 ether, address(uint160(0x1000 + i)));
}
}

function testRefundUnderflow() public {
    vm.deal(buyer, 10 ether);
    vm.deal(address(this), 10 ether);
    vm.expectRevert();
    pool.poolTransfer{value: 1 ether}(
        payable(buyer),
        5000 ether,
        "",

```

```
2,  
  
    address(0x1),  
  
    payable(feeReceiver)  
  
    );  
  
    }  
  
}
```

Impact

Public pools with many contributors can become effectively unbuyable when the gas sentinel triggers, as refunds will systematically exceed available post-fee balance and revert. Users waste gas and cannot complete purchases.

Recommendation

Set `params.returnETH = params.newPrice - params.totalETHSent` on gas break. Prefer computing fees on the actually filled amount: determine `spentNet`, set `refund = msg.value - spentNet`, distribute fees from `spentNet`, then forward seller payouts. This eliminates refund-fee mismatches.

Status

Resolved

Medium

[M-01] PublicPool# - Public pool contributor can steal mistakenly sent tokens via `contribute()` with minimal deposit

Description

`contribute()` credits the entire balance delta (`balanceOf(this) - _balance`) to the provided user. If tokens are mistakenly sent to the pool before a legitimate `contribute()` call, an attacker can contribute a minimal amount and claim ownership of the entire mistaken deposit.

Impact

Mis-sent tokens (or dust/airdrops) can be appropriated by the next contributor, leading to unintended loss for the sender and inconsistent contributor accounting.

Recommendation

Use `safeTransferFrom` inside `contribute()` to pull the exact intended amount, or enforce that `actualAmount` is within a tight tolerance of `tokenAmount`. Provide an admin-only sweep for stray tokens rather than crediting them to arbitrary contributors.

Status

Resolved

[M-02] PoolPartyLogic - PoolPartyLogic._getPublicAmount only checks reservedAmount[0], ignoring subsequent non-zero entries

Description

The guard `if(reservedAmount[0] > 0)` controls whether the loop executes. If the first entry is zero but later entries are non-zero, reservations are ignored, and all tokens go to the public.

Impact

Reserved buyers lose their allocations due to a misconfiguration.

Recommendation

Iterate over the full array unconditionally, summing all non-zero entries, or enforce `reservedAmount[0] != 0` whenever the array is non-empty during validation.

Status

Resolved

[M-03] PoolPartySwap - Arbitrary token pull from anyone who approved PoolParty contract

Description

In public pool creation (`poolType==3`), `safeTransferFrom(params.poolData[1], Pool, amount)` allows the caller to pull tokens from an arbitrary `params.poolData[1]` who has previously approved PoolParty. An attacker sets `params.poolData[1]=victim` and `params.poolData[0]=victimToken`, stealing any approved balance.

Proof of Concept

```
// SPDX-License-Identifier: MIT

pragma solidity 0.8.29;

import "forge-std/Test.sol";
import "../PoolPartySwap.sol";
import {MockERC20} from "../test/mocks/MockERC20.sol";

contract ArbitraryPullTest is Test {

    PoolParty poolParty;

    MockERC20 usdc;

    MockERC20 mockStaking;

    MockERC20 ownedPoolImpl;

    MockERC20 publicPoolImpl;

    address alice = address(0xa11ce);

    address bob = address(0xb0b);

    address feeReceiver = address(0xfee);

    function setUp() public {

        usdc = new MockERC20("USDC", "USDC", 6);

        mockStaking = new MockERC20("STAKE", "STAKE", 18);
```

```

ownedPoolImpl = new MockERC20("OPI", "OPI", 18);

publicPoolImpl = new MockERC20("PPI", "PPI", 18);


poolParty = new PoolParty();
poolParty.initialize(
    address(publicPoolImpl),
    address(ownedPoolImpl),
    address(mockStaking),
    feeReceiver,
    feeReceiver,
    feeReceiver
);


usdc.mint(alice, 10000e6);
vm.prank(alice);
usdc.approve(address(poolParty), type(uint256).max);
}

function testArbitraryPull() public {
    PoolParty.PoolCreationParams memory params = PoolParty.PoolCreationParams({
        vestingData: [uint256(0), 0, 0],
        poolType: 3,
        dataChecks: [false, false, false, false, false],
        amount: 10000e6,
        specificUsers: new address[](0),
        specificAmounts: new uint256[](0),
    });
}

```

```

        twitterUsers: new string[] (0),

        twitterAmounts: new uint256[] (0),

        poolData: [address(usdc), alice],

        priceParams: [uint256(0), 0, 0]

    });

    vm.prank(bob);

    address pool = poolParty.createPool(params);

    assertEq(usdc.balanceOf(pool), 10000e6);

    assertEq(usdc.balanceOf(alice), 0);

}
}

```

Impact

Total loss of funds for any user who has approved PoolParty to spend their tokens. This is a critical, ecosystem-wide theft vector.

Recommendation

Use `msg.sender` as the source for `safeTransferFrom` (or require `params.poolData[1] == msg.sender`). Never permit pulling tokens from arbitrary addresses during pool creation.

Status

Resolved

[M-04] PublicPool&OwnedPool - publicPool and ownedPool are UUPS but deployed as minimal clones—upgrades impossible

Description

The contracts include UUPS hooks but are deployed via `Clones.clone`, which produces minimal proxies not compatible with UUPS upgrades. Comments claim beacon-style upgrades, but no `BeaconProxy` is used. Along with that, even though there is `_authorizeUpgrade()` function, it cannot be called as there is no such function to upgrade via `PoolPartySwap`

Impact

Security fixes cannot reach existing pools; migration is required.

Recommendation

Adopt `UpgradeableBeacon` + `BeaconProxy` or remove UUPS code and plan for migrations. Update documentation accordingly.

Status

Resolved

[M-05] PoolPartyFrenlyBridge - Bridge reverts on fee-on-transfer (FOT) tokens due to mismatched amounts

Description

poolParams.amount and approvals assume no transfer tax. With FOT tokens, PoolParty pulls reverts due to insufficient actual received amount.

Impact

Bridge cannot create pools for FOT tokens; user transactions revert.

Recommendation

Use balance-delta to determine actualAmount, approve that exact amount, and set the struct accordingly.

Status

Resolved

Low

[L-01] PoolPartySwap - Incorrect error message: `_withdrawLockedFunds` reverts with `UserLocked` when user is NOT locked

Description

`if (!users[userWallet].locked) revert UserLocked();` triggers the wrong error name. It should revert with `UserNotLocked()` when the user is not locked.

Impact

Confusing error semantics hinder debugging and off-chain tooling.

Recommendation

Introduce error `UserNotLocked()` ; and use it for the negative condition.

Status

Resolved

[L-02] PoolPartySwap - CEI not followed in `_withdrawLockedFunds` (admin rug risk)

Description

The function performs an external `.call` before zeroing the user state. While admin-controlled, following CEI (checks-effects-interactions) reduces risk.

Impact

If the admin destination is a contract, reentrancy could be abused. Low likelihood

Recommendation

Zero `claimableETH` before the external call.

Status

Resolved

[L-03] PoolPartySwap - Missing zero-address validation for `partyFeeReceiver` in `updatePartyFee` & `projectReceiver` in `VerifyProject`

Description

Setting the party fee receiver to zero burns platform fees. And A Zero `projectReceiver` burns project fees.

Impact

Loss of protocol revenue until corrected. And loss of project fees until updated

Recommendation

`require(partyFeeReceiver != address(0), "Invalid fee receiver");` And also, Validate non-zero `projectReceiver` variable

Status

Resolved

[L-04] PoolPartySwap - contributeToPool docs claim support for Type 2 and 3, but code hardcodes poolType=3

Description

`contributeToPool` updates pool data with `poolType = 3`, contradicting comments that suggest it supports Type 2 as well.

Impact

Incorrect analytics and potential off-chain handling errors.

Recommendation

Clarify docs or preserve the original `poolType` when updating the struct.

Status

Resolved

[L-05] PoolPartySwap - Pause/unpause only guards claimETH; critical operations remain unpaused

Description

`whenNotPaused` is only used on `claimETH()`. Other critical operations are unaffected by pause.

Impact

Inability to halt protocol functionality during incidents.

Recommendation

Apply `whenNotPaused` to `createPool`, `buyFromPool`, `contributeToPool`, and `pool transfers`; or document the limited scope explicitly.

Status

Resolved

[L-06] PoolPartySwap - `setFrenlyBridge` missing zero-address and duplicate-address validation

Description

Setting a zero address breaks the bridge; setting the same address wastes gas.

Impact

Misconfiguration risk.

Recommendation

Validate non-zero and not-equal to the current address before setting.

Status

Resolved

[L-07] OwnedPool - `ownedPool.updateOwner` allows `newOwner = address(0)`, bricking the pool

Description

No non-zero check on `newOwner`.

Impact

Loss of pool control; functions gated by the owner break.

Recommendation

Revert on zero address with `InvalidOwnerAddress()`.

Status

Resolved

[L-08] PoolPartyFrenlyBridge - Bridge event mismatches actual influencer value when creating vaults

Description

Event `VaultCreatedForBuyer` emits `PoolParty params.influencer` while vault assigns `influencer = buyer`.

Impact

Off-chain indexers/analytics misattribute vault control.

Recommendation

Emit `buyer` as the influencer in the event.

Status

Resolved

[L-09] PoolPartySwap - Redundant `preventCancellation` call in `PoolParty.updatePoolOwner`

Description

`preventCancellation` is invoked after `updateOwner`, even though `updateOwner` already handles this parameter.

Impact

Minor gas wastage.

Recommendation

Remove the redundant call or consolidate logic.

Status

Resolved

QA

[Q-01] **PriceOracleAddressess** - 1inch oracle addresses appear outdated in [PriceOracleAddresses.sol](#)

Description

The configured 1-inch spot-price-aggregator addresses are legacy variants. The repo indicates newer deployments with added features and fixes.

Recommendation

Short term, verify current addresses per 1inch repo and update constants. Long term, make Oracle addresses configurable via timelocked governance to handle future migrations without redeploying code.

Status

Resolved

[Q-02] **ChainlinkPriceFeedRegistry** - Chainlink price feed addresses may not be the official/current ones

Description

Some Chainlink feed addresses may not match the latest documented feeds. Feeds can migrate/upgrade. If a feed is deprecated or stale, price reads could be incorrect or revert.

Recommendation

Short term, cross-validate against Chainlink's official registry and add staleness checks. Long term, make feeds updatable via governance with a timelock.

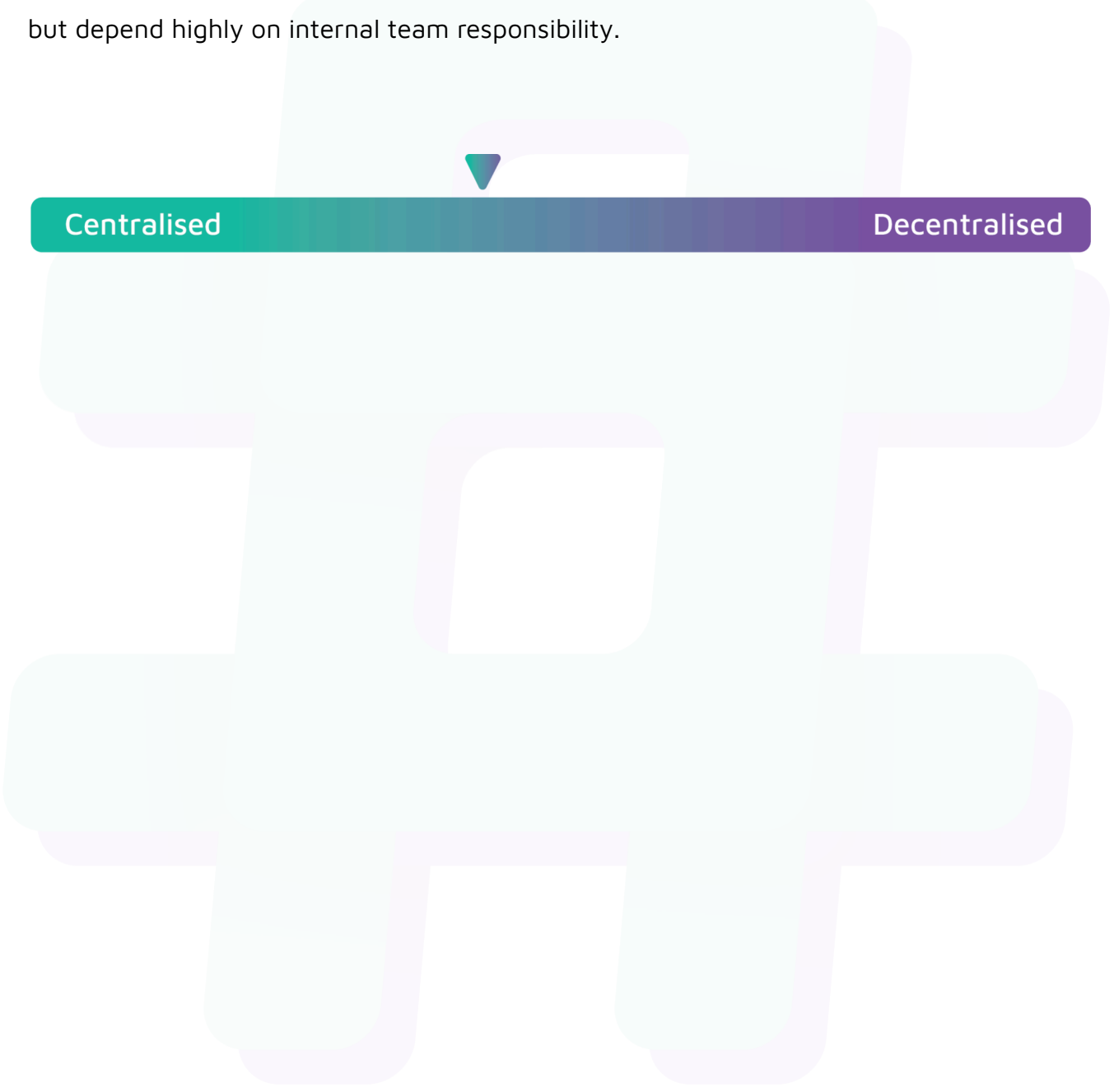
Status

Acknowledged

Centralisation

The PoolParty project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the PoolParty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

